

Seznam požadavků ke zkoušce z Datových struktur II

– Kapitoly a pojmy k ověření –

Matfyz

3. června 2026

1 Statické slovníky

1.1 Teoretický úvod a výpočetní model

- **Výpočetní model Word-RAM:**
 - Definice velikosti slova w a vztah k velikosti univerza U ($u = 2^w$).
 - Přehled operací prováděných v konstantním čase $O(1)$.
 - Paměťová omezení a adresace buněk.
- **Formulace statického slovníkového problému:**
 - Časový invariant operace **Lookup** v nejhorším případě ($O(1)$).
 - Prostorový požadavek na striktně lineární paměť ($O(n)$).

1.2 Perfektní hešování FKS (Fredman, Komlós, Szemerédi)

- **Definice c -univerzálního systému:**
- **Lemma o střední hodnotě počtu kolizí, při výběru náhodné heš. funkce:**
 - Očekávaný počet kolizí v závislosti na parametrech c , n a m .
- **Nalezení perfektní hešovací funkce h pro $m = \lceil n^2 \cdot c \rceil$ a $m = n$:**
 - Markovova nerovnost.
 - Lemma o džbánu.
 - $\mathbb{E}[\# \text{pokusů než najdeme takovou } h]$.
- **FKS algoritmus:**
 - Dvouúrovňové hešování: f, g_i .
 - $\text{Lookup}(x)$.
 - Velikost všech Q_i je $O(n)$ (trik $p_i^2 = p_i + 2\binom{p_i}{2}$).
 - Spotřeba paměti a čas na konstrukci.
- **Odbočka o třídění $x \in_R [0, 1]$:**
 - n přihrádek.
 - Bubblesort v přihrádce v $O(p_i^2)$.
 - Proč to má $O(n)$ složitost?

1.3 Deterministické slovníky

- Redukce z obecného univerza na $O(n^2)$.
- Definice q -dobré funkce
- Lemma pokud (f, g) je q -dobrá a $r \geq \log(n) + 1$, pak $\exists(f', g')$, která je q' -dobrá
 - Umím vyrobit z funkce (f, g) novou, s méně kolizema.
- Tříkrokový algoritmus, pro nalezení perfektní hešovací funkce:
 - Zvolit r . Množina $S \subset 0, 1^w$, kde $w \leq 2 \log(n) + O(1)$.
 - $(f, g) \rightarrow (f', g') \rightarrow (f'', g'')$.
 - (f'', g'') je θ -dobrá funkce.
- Zahešování x :
 - Čas $O(1)$ a prostor $O(n)$.
- Důkaz Lemmatu:
 - Seřazení řádků.
 - Volíme posuny a_i náhodně, odhadneme $\#$ nových kolizí ($\#NK$) nově posunutého řádku S_i s do teď posunutými řádky $|S_{<i}|$.
 - Zkoušíme trefit a_i do $O(1)$ pokusů (třeba 2). Ukážeme, kolik maximálně bude střední hodnota $\#NK$.
 - Všechny a_i jdou najít efektivně v čase průměrně $O(n)$.
 - Odhadneme počet všech kolizí, přes všechny řádky.
 - * Vyčistíme sumu od konce: triviální případy $|S_i| = 0$ nebo $|S_i| = 1$.
 - * Omezíme křížové součiny $|S_i| \cdot |S_{<i}|$.
 - * Využijeme odhadu $d^2 \leq 4 \cdot \binom{d}{2}$, což platí pouze pro nějaká d , k provázání $|S_j|^2$ s q , kdy q je celkový počet kolizí původních.
 - * Dosadíme do velké původní sumy pro $\#kolizí$ a dostaneme odhad $\#kolizí \leq n \cdot \lfloor 2^{3-r} \cdot q \rfloor$.
- Derandomizace:
 - Metoda podmíněné střední hodnoty.
 - Jak počítat bity $a_i = 0, 1^r$ efektivně?
 - Složitost zpracování jednoho řádku, plus složitost datové struktury, ve které si udržujeme počty prvků, v sloupečcích a podle ní počítáme bity a_i , pro řádek S_i .
 - Celková složitost derandomizovaného algoritmu $O(n \cdot \log(n))$.

2 Celočíselné datové struktury

2.1 van Emde-Boasovy stromy

- Anatomie vEB stromu:
 - $\min(S)$
 - $\max(S')$
 - Přihrádky $P_0, \dots, P_{\sqrt{U}-1}$

- Sumární strom ... *Sum*
- **Find(x)**
- **Succ(x) (nejbližší následník):**
 - Rychlá kontrola s *min*.
 - $x = (i, j)$.
 - Zkusíme přihrádku P_i .
 - Jinak vrátíme $P_{Sum.Succ(i).min}$.
- **Insert(x):**
 - Rychlá kontrola s *min*.
 - Pokud neskončíme, vezmeme x .
 - Upravíme *max* podle x .
 - $x = (i, j)$.
 - Rekurzíme se podle:
 - * Pokud P_i existuje...
 - * Pokud P_i prázdná...
- **Delete(x):**
 - Pokud prázdný strom.
 - Jinak pokud mažeme minimum, tak se může změnit x ...
 - Pak $x = (i, j)$.
 - Zavoláme $P_i.Delete(j)$.
 - Pokud se P_i vyprázdnila, musíme aktualizovat *Sum*.
 - Aktualizovat *max*, kdybychom ho smazali.
- **Problém s velikostí potřebné paměti $O(|U|)$, kde $|U| = 2^w$:**
 - Naivní řešení s obyčejným polem.
 - Trik pomocí M, I, X .
 - Operace *Read(i)* a *Write(i, y)*.

2.2 Van-Emde Boasov "intervalový" strom

- **Struktura stromu:**
 - Charakteristický vektor S .
 - $O(|U|)$ vrcholů.
 - Hloubka $O(\log |U|) = O(w)$.
 - Monotónnost cesty z listu do kořene.
- **Operace:**
 - Rychlost operací *Pred(x)*, *Succ(x)*, *Find(x)*, pokud budeme umět rychle bin. vyhledávat na cestě z listu do kořene, bude $O(LL(|U|))$, jinak $O(L|U|)$.
 - Plus musíme udržovat *min* a *max* podstromů a spojový seznam prvků S , abychom zvládli i situaci, kdy se ptáme na *Pred/Succ* prvku x , který je v množině S .
 - Problém s *Updaty*, běží v $O(L|U|)$.

2.3 X-fast trie

- **Struktura stromu:**
 - Vychází z "intervalového" stromu + kukaččí hešování.
 - Ukládáme pouze vrcholy, které jsou potřeba, tedy vrcholy odpovídající prefixům prvků, které jsou v S .
 - Vyhledáváme na cestě mezi listem a kořenem binárně, pomocí dotazu v $O(1)$, díky hešování.
 - Místo neexistujících synů nějakého vrcholu, si uložíme odkaz na levého/pravého následníka (list) v jeho podstromě, který existuje.
- **Operace:**
 - Operace *Find*, *Pred*, *Succ* běží v čase $O(LL(|U|))$.
 - Operace *Insert*, *Delete*, běží v čase $O(L(|U|))$ průměrně amortizovaně.
- **Paměťová složitost** $O(n \cdot \log(|U|))$

2.4 Y-fast trie

- **Struktura stromu**
 - Setříděné prvky rozdělíme na bloky velikosti $B = \Theta(\log(|U|))$.
 - Každý blok svého reprezentanta (typicky největší x). Uvnitř bloků BVS
→ operace v $O(L(|B|)) = O(LL(|U|))$.
 - Globální úroveň je x-fast trie a do ní ukládáme pouze reprezentanty těchto BVS.
 - Prvků v x-fast trii je $O(n/\log(|U|))$.
- **Paměťová složitost:**
 - Všechny BVS dohromady $O(n)$.
 - Globální x-fast trie $O(\#prvků \cdot \log(|U|)) = O(n)$.
- **Operace:**
 - *Find*(x), *Succ*(x), *Pred*(x) - globální strom mi řekne následníka a předchůdce, tedy dva BVS a v nich opakují dotaz na x .
 - Složitost $O(L(B)) = O(LL(|U|))$.
- **Updaty:**
 - Normálně probíhají, bez reorganizace globální x-fast trie
 - Udržíme invariant hustoty bloku $[\frac{1}{4}B, 1B]$. Jakmile blok podteče/přeteče tak slučujeme/dělíme.
 - Amortizovaně složitost $O(1)$. Dokazujeme skrz složitost globální x-fast trie vs. jak často se to děje?
 - **POZOR** na jednu věc u mazání.

3 Haldy

3.1 Srovnání hald

- **Binární vs. k -regulární halda**
 - Složitosti operací *Insert*, *Min*, *ExtractMin*, *Decrease*, *Delete*, *Build*, *Merge*.
 - Dijkstruv algo. používá $n \times \text{Insert}$ a $n \times \text{ExtractMin}$ a $m \times \text{Decrease}$.
 - Pro k -regulární haldu, celková složitost minimální pro $k := \frac{m}{n}$.
 - Binomiální umí navíc *Merge* v $O(\log(n))$.

3.2 Binomiální halda

- **Binomiální strom B_k :**
 - Dvě rekurzivní definice.
 - Pozorování na počet vrcholů, počet hladin a maximální stupeň kořenu, což je i maximální stupeň ve stromě.
- **Binomiální halda:**
 - Definice: Les binomiálních stromů, v každém vrcholu jeden prvek, platí haldové uspořádání.
 - Počtem prvků je jednoznačně určeno, které binomiální stromy v posloupnosti existují.
 - Do $O(\log(n))$ se vleze:
 - * řády stromů,
 - * # stromů v lese,
 - * hloubky,
 - * maximální # dětí.
 - Uložení v paměti, každý vrchol:
 - * prvek + priorita,
 - * řád (počet dětí),
 - * ukazatel na rodiče,
 - * ukazatel na nejlevější dítě,
 - * ukazatel na levého/pravého sourozence (zapojení cyklicky, pokud jsou to kořeny tak...)
- **Operace:**
 - *Min* v $O(\log(n))$
 - *Merge* v $O(\log(n))$
 - *Insert* v $O(\log(n))$
 - *Build* v $O(n)$
 - *ExtractMin* v $O(\log(n))$
 - *DecreaseKey* v $O(\log(n))$
 - *IncreaseKey* v $O(\log(n))$
 - *Delete* v $O(\log(n))$

3.3 Líná binomiální halda

- Myšlenka lenosti.
- Složitost operací *Merge*, *Insert* a *Build*.
- Analýza *ExtractMin*, složitost $O(t + \log(n))$, kde $t := \# \text{stromů v lese}$.
 - Správně si zvolit potenciál.

3.4 Fibonacciho Halda

- Struktura haldy "Jak funguje, černý a bílý vrchol."
- Operace *Insert*, *Merge*, *Build*.
- Lemma 0 o Fibonacciho číslech:
 - Rekurzivní definice Fibonacciho čísla.
 - Jiná přímá definice Fibonacciho čísla, kterou uvádí Lemma.
 - Fibonacciho číslo roste exponenciálně!
- Lemma* o Tloušťce stromu:
 - Vrchol s k dětmi má. . .
 - Pomocné tvrzení v důkazu o řádech dětí nějakého vrcholu: $\forall i : r_i \geq i - 2$.
 - * . . .
 - Důkaz Lemma*, které
- Operace *Decrease* a *Cut*.
- Těžká práce u *ExtractMin*.

4 Splay stromy

4.1 Analýza Splay stromů

- Opakování Splay Stromů
- Věta Access Time:
 - Zadefinování pojmů jako $T(v)$, $s(v)$, $r(v)$, $\Phi(v)$.
 - Znění: Amortizovaná cena $\text{Splay}(x) \leq 3(r'(x) - r(x)) + 1$, což je v $O(\log(n))$.
 - Přepis $A_i = R_i + (\Phi_i - \Phi_{i-1})$ pro $i \in [m]$, za použití teleskopické sumy, kdy chceme odhadnout R_i .
- Věta $m \times \text{Splay}$ trvá $O(m \cdot \log(n) + n \cdot \log(n)) \dots$
- Váhová analýza:
 - Definice vah, pro vrcholy $w(v)$ a nové velikosti $s(v)$.
 - Pozorování 1, že věta Access Time stále platí.
 - Pozorování 2, $3(r'(x) - r(x)) + 1$ se už nevejde do $O(\log(n))$, ale je potřeba lépe odhadnout, pomocí: $r_{\max}$ a $r_{\min}$.
- Analýza operací *Find*(x), *Insert*(x) a *Delete*(x)

- Všechny se vlezou do $O(r_{max} - r_{min} + 1)$.
- **Lemma W**
 - Vzorec pro to, jak dlouho trvá $Splay(x)$, pokud známe váhu x a celkovou váhu stromu.
 - Důkaz odhadneme $O(r_{max} - r_{min} + 1)$.
- **Váhovou teorii s Lemmatem W aplikujeme na úplně obyčejný Splay strom, kde žádný prvek neupřednostňujeme, vyjde nám ta stará známá složitost $O(\log n)$?**
 - Nastavení vah rovnoměrně náhodně.
 - Dosazení do Lemmat.
 - Celková cena m operací.
 - Proč můžeme předpokládat, že $W = 1$?
- **Staticky optimální strom.**
- **Věta o Statické optimalitě**
 - Znění, že Splay strom zpracuje asymptoticky sekvenci dotazů..
 - Důkaz
 - * $w(x) = 3^{-c_T(x)}$
 - * Celková váha $W < 1$
 - * Dosazení do Lemmatu W
 - * Vyúčtování sekvence m operací, tedy $\Phi_0 - \Phi_m$.
- **Static Finger bound ... PRESKAKUJI**
- **Working Set Bound**
 - Co to je Working set a velikost této množiny z_i .
 - Znění věty.
 - LRU seznam a že pozice v něm prvku x je $p(x) = z(x)$.
 - Nastavíme váhy $w(x) = \frac{1}{(p(x)+1)^2}$.
 - Ukážeme, že W , je opět omezeno konstantou, tedy v $O(1)$.
 - Vyúčtování $\Phi_0 - \Phi_m$ - NECHAPU.

5 Cestové dotazy a updaty na stromech

5.1 Cesta

- **Motivace - kostry!**
- **Intervalový strom nad cestou**
- **Všechny cestové dotazy i updaty v $O(\log(n))$, pomocí kanonických podstromů a líného propagování změny.**

5.2 Heavy-Light dekompozice (Strom)

- **Definice HLD, definice těžké hrany.**
- **3 Pozorování:**
 - Každý vrchol má maximálně 1 těžkou hranu
 - Z toho plyne, že těžké hrany tvoří disjunktní cesty $\rightarrow$ můžeme na ně použít cestovou dekompozici.
 - Každá cesta kořen-list obsahuje maximálně $O(\log(n))$ hran.
- **Konstrukce pomocí dvou DFS v $O(n)$.**
- **LCA v HLD v $O(\log(n))$**
 - Projdu po lehkých hranách.
 - Pokud narazím na stejnou těžkou cestu $\rightarrow$, tak беру z těch dvou vyšších vrcholů, což v $O(1)$, díky tomu, co si každá těžká cesta pamatuje.
- **Cestový dotaz/update**
 - Najdeme $LCA(u, v)$, a rozdělíme dotaz na dvě cesty.
 - Na každé navštívíme max $O(\log(n))$ lehkých hran a těžkých cest
 - Každý dotaz na těžké cestě v $O(\log(n))$, celkově tedy $O(\log^2(n) + \log(n)) = O(\log^2(n))$.

5.3 Link-Cut Stromy

- **Struktura LCS a rozhraní:**
 - Tlusté a tenké hrany.
 - Strukturální operace: $Root(v)$, $Parent(v)$, $Cut(v)$, $Link(u, v)$, $Evert(v)$.
 - Hodnotové operace: $Cost(v)$, $PathMin(v)$, $SetCost(v)$, $PathUpdate(v, \delta)$.
- **Mikro-architektura: Splay Strom pro jednu tlustou cestu**
 - Splay strom = tlustá cesta.
 - Inorder průchod SS je pořadí na TC ze spoda nahoru.
 - Líné vyhodnocování: vrcholy SS si udržují minimum podstromu, $+\delta$ a $ReverseFlag$
- **Makro-architektura**
 - Kořen Splay stromu $\neq$ nejvyšší vrchol na příslušné tlusté cestě v LCS.
 - Kořen SS si pamatuje odkaz na svého lehkého rodiče.
 - Jeden LCS je jeden ze stromů, který se snažíme reprezentovat.
- **Líné vyhodnocování**
 - Při průchodu zespoda nahoru vytvoříme zásobník, pak už jen projdeme zásobník a tím procházíme od shora dolů.
- **Globální pohled + Operace $Expose(v)$**
- **Jak funguje $Expose(v)$ + amortizace... strany 39-41**
- **Aplikace na Dinicově algoritmu. - PRESKAKUJI**

6 Dynamizace

6.1 Dynamizace dvojrozměrných intervalových stromů

- **Dvojrozměrný intervalový strom**
 - Struktura stromu, primární strom a sekundární stromy.
 - Obdelníkové dotazy v čase $O(\log^2(n))$.
 - Velikost stromu je pouze $O(n \cdot \log(n))$.
 - Budování stromu v čase $O(n \cdot \log(n))$.
- **Líné vyvážování**
 - Vložení bodu.
 - Udržíme α vyváženost.
 - Částečný rebuild, jakmile je někde podmínka porušena.
 - Amortizovaná cena $\log^2(n)$.

6.2 Obecná dynamizace

- **Definice: vyhledávací problém**
 - $f : U_Q \times P_{fin}(U_X) \rightarrow U_R$
- **Rozložitelnost: klíčová vlastnost**
 - $\exists \square : U_R \times U_R \rightarrow U_R$, vyčíslitelná v konstantním čase, taková, že...
 - Tři příklady rozložitelných a nerozložitelných problémů.
- **Výchozí bod - Statická struktura, až SemiDynamizace a Dynamizace**
 - Statická struktura - $B_S(n), Q_S(n), S_S(n)$ + rozumné chování těchto složitostí.
 - Semidynamická struktura - $Q_D(n)$ (w.c.), $S_D(n), \overline{I_D(n)}$ (amortizovaně). Pokud chceme dynamickou, ještě musíme přidat delete v čase $\overline{D_D(n)}$ amortizovaně.
 - Struktura zdynamizování Statické.
 - *Query* - zeptáme se každého bloku a zkombinujeme a jeho $Q_D(n)$.
 - *Insert* - Najdeme nejbližší prázdný blok B_i , rozložíme bloky $< i$. Složíme do bloku B_i .
 - Odvození, že $S_D(n) \leq S_S(n)$.
 - Důkaz amortizované složitosti $\overline{I_D(n)}$.
- **Worst-Case Dynamizace (STRANA 49)**
 - Zlepšení těch dlouhých operací *Insert*, kdy dojde k přestavbě.
 - Životní cyklus bloků, vždy žijou dva bloky od jednoho řádu i a building bloku $i + 1$ delame prubezne.
 - Dotaz stále v $Q_D(n) \in O(Q_S(n) \cdot \log n)$
 - Vkládání má novou w.c. složitost $I_D(n) \in O(\frac{B_S(n)}{n} \cdot \log n)$.
- **Úplná dynamizace $\rightarrow$ přidání *Delete***
 - Pomocný adresář, pro pamatování, ve kterém bloku prvek je.
 - Místo mazání prvku děláme pomníčky.
 - Jakmile moc pomníčků $\rightarrow$ re-build.
 - Amortizovaná cena $\overline{D_D(n)}$ je $O(\frac{B_S(n)}{n} \cdot \log n)$

7 Perzistence

7.1 Typy Perzistence

- **Efemérní, Semiperzistentní, Plně perzistentní, Funkcionální struktury.**
- **Funkcionální seznamy s prependem - STRANA 51**
 - Jednosměrný spojový seznam s přidáváním na začátek.
- **Funkcionální BVS - STRANA 51**
 - Kopírujeme cestu do kořene, verze je prostě odkaz na nový kořen.
 - Paměť na verzi $O(\log n)$ a čas $O(\log n)$.
- **Aplikace funkcionálních BVS - Lokalizace bodu v rovině rozdělené polygony STRANA 52 - 53**
 - První řešení, pomocí dvou seřazených struktur, podle pásů a úseček v pásu $\rightarrow$ prostor $\in O(n^2)$, vybudování $\in O(n^2)$, dotaz $\in O(\log n)$.
 - Lepší řešení pomocí funkcionálního BVS, kdy zametáme rovinu zeshora a vždy událost je přidání, odebrání úsečky. Pro každou událost postavíme verzi a každý pás úseček je pak tento funk. BVS $\rightarrow$ dotaz stále $\in O(\log n)$, prostor $\in O(n \log n)$, vybudování $\in O(n \log n)$.

7.2 Semi-Persistentní DS

- **Pointerové DS**
 - Krabíčka - má $O(1)$ dat, $O(1)$ ukazatelů. Držátko $O(1)$ ukazatelů.
 - Operace co umíme s takovou DS: čtení krabíčky, skok na jinou krabíčku, modifikace krabíčky, zakládání krabíčky.
- **Co vše musíme verzovat? Strukturální změny vs nestrukturální**
- **První technika: Tlusté vrcholy**
 - Vrchol si pamatuje celou svoji historii.
 - (počáteční vrchol, verze, změny)
 - Update v $O(1)$. Ale omezený počet verzí.
 - Query se zpomalí na $O(\log^2 n)$, kvůli binárnímu vyhledávání správné verze vrcholu.
- **Obecná semi-perzistentní DS**
 - Graf má omezený vstupní vrchol $\leq p$.
 - Anatomie krabíčky: Počáteční verze, data, žurnál (e slotů změn, platí $p \leq e$), zpětné pointery.
 - *Query* - přijdu do krabíčky, aplikuji všechny změny až do aktuální verze, těch je max e , což znamená, že konstatně mnoho, tedy v $O(1)$.
 - *Update* - pokud volno, prostě připišeme změnu a jinak musíme přetečenou krabíčku kopírovat a musíme změnit zpětné pointery rodičů na tuhle aktuální krabíčku.
 - Trik na zacyklení, abychom nekopírovali do nekonečna.
 - Důkaz amortizace, že každá strukturální změna se děje v $O(1)$ amortizovaně.

- **Úplná perzistence DS**

- Problém s amortizovanou složitostí, můžeme se totiž vracet do verze, kdy došlo k drahému kopírování a vždy ho vynutit.
- Problém s porovnáním verzí, což se dá řešit projitím stromu "kolem", tedy linearizace historie.
- Musíme navíc omezit konstantou výstupní stupeň.
- Zbytek složitý...

8 Cache struktury

8.1 List Labeling problem

- **Definice problému. Jinými slovy List Order.**

- **LL v exponenciálním rozsahu**

- $\#$ prvků $\leq M$.
- Zarážky jsou na pozicích $0, 2^M$.
- Nový prvek dám na průměr sousedů.
- Po M insertech mezery aspoň 1.
- $O(1)$ na insert w.c.
- Funguje pouze pro $M \in O(w)$, kde w je šířka slova.

- **LL v polynomiálním rozsahu**

- Používáme bin. strom hloubky $O(\log M)$.
- Externí vrcholy jsou prvky.
- Kroky ve stromě L/P jsou bity v labelu.
- Dřívější prvky dostanou nižší labely.
- $\#$ bitů labelu je $O(\log M)$.
- *Insert* znamená změnit příslušný externí vrchol na interní a pod něj připojit přelabelovaný prvek a nový prvek.
- Vyvažovat budeme líně s BB- α stromy.
- Přebudování stojí $O(k)$, kde k je $\#$ prvků podstromu a musíme je přelabelovat, to se ale schová do $O(k)$ a tedy amortizovaně insert běží jako v líně vyvažovaném, tedy $O(\log n)$.

- **Finální List Order v $O(1)$ amortizovaně**

- Rozdělíme posloupnost na bloky velikosti $\theta(\log n)$
- Uvnitř bloků používáme labelování v exponenciálním rozsahu.
- Nad bloky používáme labelování v poly.
- Prvek má (label bloku, label uvnitř bloku).
- Dokud *Insert* do bloku, tak v $O(1)$.
- Jakmile přeteče, tak ho rozdělíme na dva poloviční a přidáme blok v globálních blocích, tedy musíme přelabelovat všechno uvnitř jednoho bloku.
- Čas $O(\log n)$ na přelabelování, čas na globální *Insert* $O(\log n)$ amortizovaně. Amortizovaně $O(\frac{1}{\log n})$ krát.
- ZRADA, musí se přidat indirekce, kvůli přelabelování ostatních bloků při velkém globálním rebuildu, ve kterých je $O(\log n)$ prvků.

8.2 Van-Emde-Boasovo uložení binárního stromu

- Opakování Cache modelu, M , B , cache-aware je C/A , cache-oblivious je C/O .
- Uložení úplného bin. stromu s hloubkou $h = 2^k$.
 - Rozřízneme v $h/2$.
 - Uložíme nejdříve vrchní strom, poté těch hodně nižších zleva doprava.
 - Pro paždý podstrom se rekurzíme.
 - Zascopujeme na hloubku rekurze i , kde už se $S_i \leq B$, nad touto hloubkou rekurze je $S_{i-1} > B$ a taky $S_{i-1} = S_i^2$.
 - Z toho plyne, že na této hloubce je $\sqrt{B} < S \leq B$.
 - Hloubka S je tedy $\frac{1}{2} \log B < \log S \leq \log B \implies \log S \in \Theta(\log B)$.
 - Každý strom v tomto scopu se načte do cache v 1 až 2 operacích $\implies O(1)$ práce.
 - # kusů je $O(\frac{\log n}{\log B})$.
 - $\implies$ cesta z kořene do listu stojí $O(\frac{\log n}{\log B}) \cdot O(1) = O(\frac{\log n}{\log B})$
- Uložení úplného bin. stromu s obecnou hloubkou h .
 - Problém je, že při určování hloubek horního a spodních stromů musíme zaokrouhlovat nahoru nebo dolů $(+1) \rightarrow$ může se počítat blbě.
 - Trik: $h_b :=$ nejbližší mocnina 2 vzhledem k h , $h_t := h - h_b$.
 - Lze ukázat, že poté pouze první strom, úplně nahoře bude anomálně velký a jinak ostatní už pěkně mocnina 2.

8.3 Packed Memory Array

- Struktura
 - Mám pole s mezerami, rozdělím na kusy velikosti $\Theta(\log n)$, kdy # kusů $= 2^h$, h je hloubka bin. stromu, který nad kousky stavím.
 - hustota intervalu $:= \frac{\# \text{ prvků}}{\text{kapacita}}$, standartní hustota v hloubce i je $\in [\frac{1}{2} - \frac{i}{4h}, \frac{3}{4} + \frac{i}{4h}]$.
- $Insert(x)$
 - Přidám ho do kousku, pokud přeteče, najdu na cestě kousek-kořen první vrchol, který nepřetekl přes svou std. hustotu.
 - $\forall$ prvky přerozdělím rovnoměrně pod tímto vrcholem. Pozorování, některé vrcholy nad tímto vrcholem budou stále přeteklé nad std. hust., ale to nevádí, protože hustota vrcholu je aritmetický průměr potomků na jedné hladině.
 - Pokud dojdou až do kořene, proběhně celostromová reorganizace.
- Amortizovaná cena $Insert(x)$
 - Provádíme reorganizaci vrcholu, který $\rho \leq \frac{3}{4} + \frac{i}{4h}$ a $\exists$ dítě $d : \rho_d > \frac{3}{4} + \frac{i}{4h}$.
 - Pokles hustoty aspoň o $\frac{1}{4h}$. Pokud má rodič kapacitu k , tak ubude aspoň $(\frac{1}{4h}) \cdot (\frac{k}{2}) = \frac{k}{8h} \stackrel{\circ}{=} \frac{k}{\log n}$ prvků.
 - Preorganizace intervalu délky k trvá $O(k)$, proto amort. cena $Insert(x)$ je $O(\log^2(n))$, protože ještě musíme sečíst přes všechny hladiny.
- Reorganizace celého pole

- Musíme zvětšit pole tak, aby hustota v novém kořeni vyšla mezi $[\frac{1}{2}, \frac{3}{4}] \rightarrow$ mezi další globální reorg. musí proběhnout lineárně mnoho $Insert(x)$.
- Počet nových kousků musí být mocnina 2.

- **Jak počítat hustoty?**

- Spočítáme hustotu, tedy počet prvků, jednoho nalezeného kousku. $\dots \Theta(\log n)$
- Přidáme levý/pravý sousedící interval dvakrát delší délky.
- Opakujeme, dokud nepokreujeme interval velikosti k .
- Stojí to $\Theta(k)$, reorganizace těchto prvků stojí $O(k)$, proto se to zaplatí.

- **I/O složitost: počítání hustot a reorganizace**

- Čtení kousků, pro počítání hustoty jde pomocí dvou skenů, jeden $\leftarrow$ a druhý $\rightarrow$, kdy ty stále dvojnásobné intervaly, které přidávám vpravo čtu pomocí $\rightarrow$ a vice-versa.
- Reorganizace: 1. ztūkám prvky doprava v intervalu, který reorganizujeme, pomocí dvou skenů, kdy jeden čte, druhý zapisuje. 2. rozházím ztukané prvky pomocí dvou skenů, kdy oba stejný směr a jeden čte a druhý zapisuje. Nikdy nenarazím na ztukanou část.
- Max šest skenů $\Rightarrow \Theta(\frac{k}{B} + 1)$
- Amortizovaná I/O složitost bude $O(\frac{\log^2 n}{B} + 1)$, protože Insert stojí amortizovaně tolik a I/O operací je B krát méně.

- **Udržování vEB bin. stromu nad PMA**

- Musíme přepočítat klíče v předcích prvků, které reorganizujeme.
- Celková cena $O(\frac{\log n}{\log B} + \frac{k}{B} + \frac{\log k}{\log B})$.
- $\frac{\log n}{\log B}$ za cestu z LCA (krajů reorganizovaného intervalu) do kořene, to ale stejně potřebujeme na nalezení kusu, do kterého budeme vkládat.
- $\frac{k}{B}$ za všechny stromečky, co jsou uvnitř, vleze se do $O(\frac{k}{B} + 1)$, tedy amortizovaně $\frac{\log^2 n}{B} + 1$.
- $\frac{\log k}{\log B}$ za stromečky, co jsou na krajích, za každou hladinu max dva, to se ale vleze do $O(\frac{k}{B} + 1)$.
- Celková amortizovaná složitost $Insert$ včetně udržování vEB bin. stromu nad je $O(\frac{\log^2 n}{B} + \frac{\log n}{\log B} + \frac{\log k}{\log B})$

- **Indirekce - Zbavení se kvadrátu v logaritmu**

- Vytvoříme fragmenty, ve kterých budou uloženy prvky, velikost fragmentu $\Theta(\log n)$.
- Do původní struktury si ukládám pouze reprezentanty.
- Najdu reprezentanta, tím najdu fragmenty (stojí $O(\frac{\log n}{\log B})$), do kterých prvek vkládám, pokud přeteče, tak řeším rozdělením fragmentu na dva a ten druhý vkládám.
- Tím operace globální operace $Insert_G(fragment)$ a ta nastane velikost fragmentu $:= \log n$ krát méně často, tedy amortizovaně proběhne při každém $Insert_u \frac{1}{\log n}$ operací globálního $Insert_u$.
- Fragment je souvislá část paměti, operace s nimi v $O(\frac{\log n}{B})$.
- Amortizovaně jeden $Insert$ bude v $O(\frac{\log n}{B} + 1)$ I/O složitosti.
- $Find$ bude v $O(\frac{\log n}{\log B})$ I/O složitosti.

9 Úsporné datové struktury

9.1 Úvod

- C je mn. objektů, min. počet bitů na uložení je $\text{OPT} = \lceil \log |C| \rceil$.
- Implicitní, Úsporné, Kompaktní DS.
- Reprezentace řetězce nad abecedou Σ .
 - $\Sigma = [10] \rightarrow \lceil \log 10 \rceil = 4$ bity.
 - Pokud budeme kódovat po dvojici, tak $\lceil \log_2 100 \rceil = 7$.
 - Obecně pro k znaků můžeme uložit do $k \cdot \log_2 10 + 1$ bitů, lepší než $k \cdot (\log_2 10 + 1)$.
 - ALE zrušíme si lokální dekódovatelnost.
- $\text{Redundance} := \text{Skutečně zabraný prostor} - \text{OPT}$.

9.2 SOLE - Bezprefixové kódy

- Naivní přístup:
 - Zprávu na bloky velikosti b .
 - Padding 10000...na konec zprávy, aby byla dělitelná b .
 - Za každý blok bit 0 nebo 1, zda zpráva pokračuje, či končí.
 - $\text{Redundance} = b + \lceil \frac{n}{b} \rceil = b + \frac{n}{b} + 1$.
- SOLE:
 -